Desarrollar software a partir de la integración de sus componentes

Dilan Giovanny Molina Noy

Centro de servicios financieros, Servicio Nacional de Aprendizaje

SENA

3118311: Análisis y desarrollo de software

Andrés Rubiano Cucarían

19 de marzo de 2026

I. Introducción

Este documento presenta el desarrollo e implementación de los módulos principales de una aplicación web orientada a la gestión de una farmacia, denominada *El Remedio del Sol*.

La aplicación fue desarrollada utilizando tecnologías modernas como **Node.js, Express, MongoDB y React**, siguiendo una arquitectura cliente-servidor.

Se implementaron funcionalidades como autenticación mediante JWT, gestión de productos, gestión de citas y un dashboard interactivo, aplicando buenas prácticas de desarrollo, control de versiones y pruebas básicas del sistema.

II. OBJETIVOS

2.1. Objetivo General

Desarrollar y codificar los módulos de una aplicación web para la gestión de una farmacia, cumpliendo con los requerimientos del sistema y aplicando buenas prácticas de desarrollo.

2.2. Objetivos Específicos

- Implementar autenticación de usuarios mediante JWT.
- Desarrollar CRUD de productos.
- Desarrollar CRUD de citas.
- Diseñar la interfaz de usuario con React.
- Implementar conexión con base de datos MongoDB.
- Aplicar control de versiones con Git.

III. METODOLOGÍA DE DESARROLLO

Se utilizó una metodología ágil basada en **desarrollo incremental**, permitiendo construir el sistema por módulos:

- Módulo de autenticación
- Módulo de productos
- Módulo de citas
- Dashboard

Cada módulo fue desarrollado, probado y posteriormente integrado.

IV. ARQUITECTURA DEL SISTEMA

La aplicación se basa en arquitectura de **tres capas**:

Frontend

- React
- Manejo de rutas
- Consumo de API

Backend

- Node.js + Express
- API REST
- Controladores y rutas

Base de Datos

- MongoDB
- Modelos con Mongoose

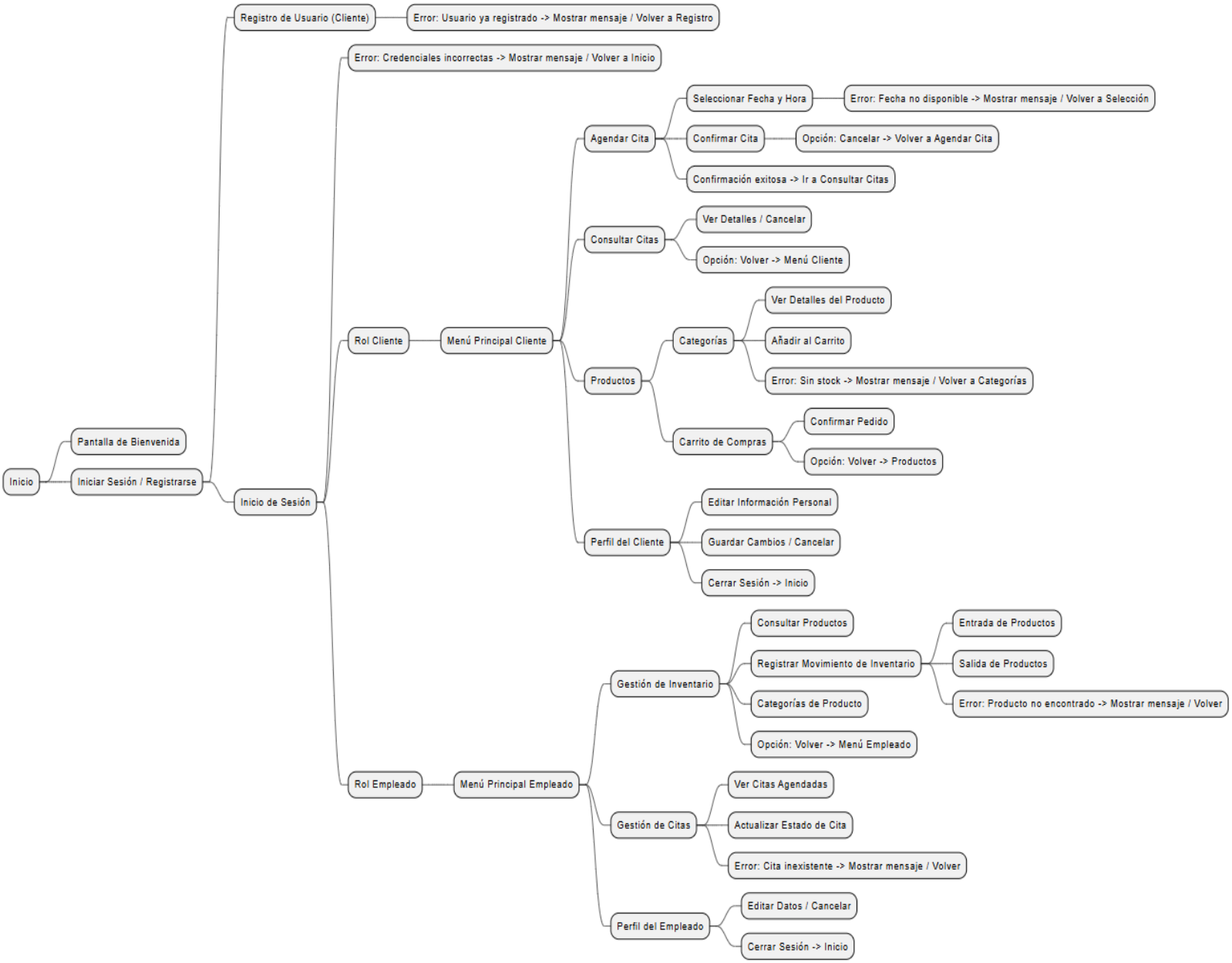
V. REQUERIMIENTOS DEL SISTEMA

- RF1: El sistema debe permitir registrar usuarios.
- RF2: El sistema debe permitir iniciar sesión con autenticación JWT.
- RF3: El sistema debe permitir crear, editar y eliminar productos.
- RF4: El sistema debe permitir gestionar citas.
- RF5: El sistema debe mostrar un dashboard con acceso a módulos.
- RF-06: El sistema mostrará alertas de stock mínimo.
- RF-07: El sistema permitirá cancelar o reprogramar una cita desde la app
- RF-8: El sistema debe registrar en un log seguro todas las acciones relevantes incluyendo: inicio de sesión, creación/modificación de citas, cambios de inventario, eliminaciones y accesos no autorizados.

VI. Ejemplo de algunas de las Historias de Usuario

- HU1: Como usuario, quiero iniciar sesión para acceder al sistema.
- HU2: Como administrador, quiero gestionar productos.
- HU3: Como usuario, quiero agendar citas.
- HU4: Como usuario, quiero ver mis citas registradas.

VII. Mapa de navegación



VIII. DESARROLLO DE LOS MÓDULOS

8.1. Módulo de Autenticación

- Login con JWT
- Registro de usuarios

Ejemplo de código

```
JS auth.middleware.js ×
src > middlewares > JS auth.middleware.js > ...
1   import jwt from "jsonwebtoken";
2
3   export const verificarToken = (req, res, next) => {
4     const token = req.headers["authorization"];
5
6     if (!token) {
7       return res.status(401).json({ msg: "Acceso denegado" });
8     }
9
10    try {
11      const decoded = jwt.verify(token, process.env.JWT_SECRET);
12      req.usuario = decoded;
13      next();
14    } catch (error) {
15      res.status(401).json({ msg: "Token inválido" });
16    }
17  };
```

8.2. Módulo de citas

Funcionalidades:

- Agendar cita
- Listar citas
- Eliminar cita

Ejemplo de código

farmacia-remedio-sol > src > pages > JS Citas.js > Citas > agendar

```
import React, { useState, useEffect } from "react";
import { crearCita, getCitas, eliminarCita } from "../services/citaService";

function Citas() {

  const [fecha, setFecha] = useState("");
  const [hora, setHora] = useState("");
  const [motivo, setMotivo] = useState("");
  const [citas, setCitas] = useState([]);

  useEffect(() => {
    cargarCitas();
  }, []);

  const cargarCitas = async () => {
    const data = await getCitas();
    setCitas(data);
  };

  const agendar = async () => {
    try {
      const fechaCompleta = new Date(`${fecha}T${hora}`);

      const data = await crearCita({
        fecha: fechaCompleta,
        motivo
      });

      if (data._id) {
        alert("Cita agendada correctamente");

        setFecha("");
        setHora("");
        setMotivo("");

        cargarCitas();
      }

    } catch (error) {
      console.error(error);
      alert("Error al crear la cita");
    }
  };

  const eliminar = async (id) => {
    await eliminarCita(id);
    cargarCitas();
  };

```

8.3. Módulo de Productos

Funcionalidades:

- Crear producto
- Editar producto
- Eliminar producto
- Listar productos

Ejemplo de código:

```
3
4   function Productos() {
5
6     const [nombre, setNombre] = useState("");
7     const [precio, setPrecio] = useState("");
8     const [stock, setStock] = useState("");
9     const [productos, setProductos] = useState([]);
10    const [editando, setEditando] = useState(null);
11
12    useEffect(() => {
13      cargarProductos();
14    }, []);
15
16    const cargarProductos = async () => {
17      const data = await getProductos();
18      setProductos(data);
19    };
20
21    const guardar = async () => {
22
23      if (editando) {
24        await actualizarProducto(editando, {
25          nombre,
26          precio,
27          stock
28        });
29
30        alert("Producto actualizado");
31        setEditando(null);
32
33      } else {
34        const data = await crearProducto({
35          nombre,
36          precio,
37          stock
38        });
39
40        if (data._id) {
41          alert("Producto creado");
42        }
43      }
44
45      setNombre("");
46      setPrecio("");
47      setStock("");
48
49      cargarProductos();
50    };
51
```

8.4. Dashboard

- Navegación entre módulos
- Interfaz amigable
- Uso de componentes reutilizables

Ejemplo de código:

```
farmacia-remedio-sol > src > pages > JS Dashboard.js > Dashboard
1   import React from "react";
2   import { Link } from "react-router-dom";
3
4   function Dashboard() {
5
6     return (
7
8       <div>
9
10        <h2>Bienvenido al sistema</h2>
11        <p>Selecciona una opción del menú</p>
12
13        <div className="dashboard-grid">
14
15    <Link to="/productos" className="dashboard-card card-productos">
16      <h3>💊 Productos</h3>
17      <p>Gestionar medicamentos</p>
18    </Link>
19
20    <Link to="/citas" className="dashboard-card card-citas">
21      <h3>📅 Citas</h3>
22      <p>Administrar citas</p>
23    </Link>
24
25    <Link to="/" className="dashboard-card card-usuarios">
26      <h3>👤 Usuarios</h3>
27      <p>Configurar perfil</p>
28    </Link>
29
30  </div>
31
32      </div>
33
34    );
35  }
36
37  export default Dashboard;
```

IX. Conexión con base de datos mongo

Se realiza la conexión con la base de datos mongo db mediante un archivo en el que se cita la uri de la base de datos

Ejemplo de código:

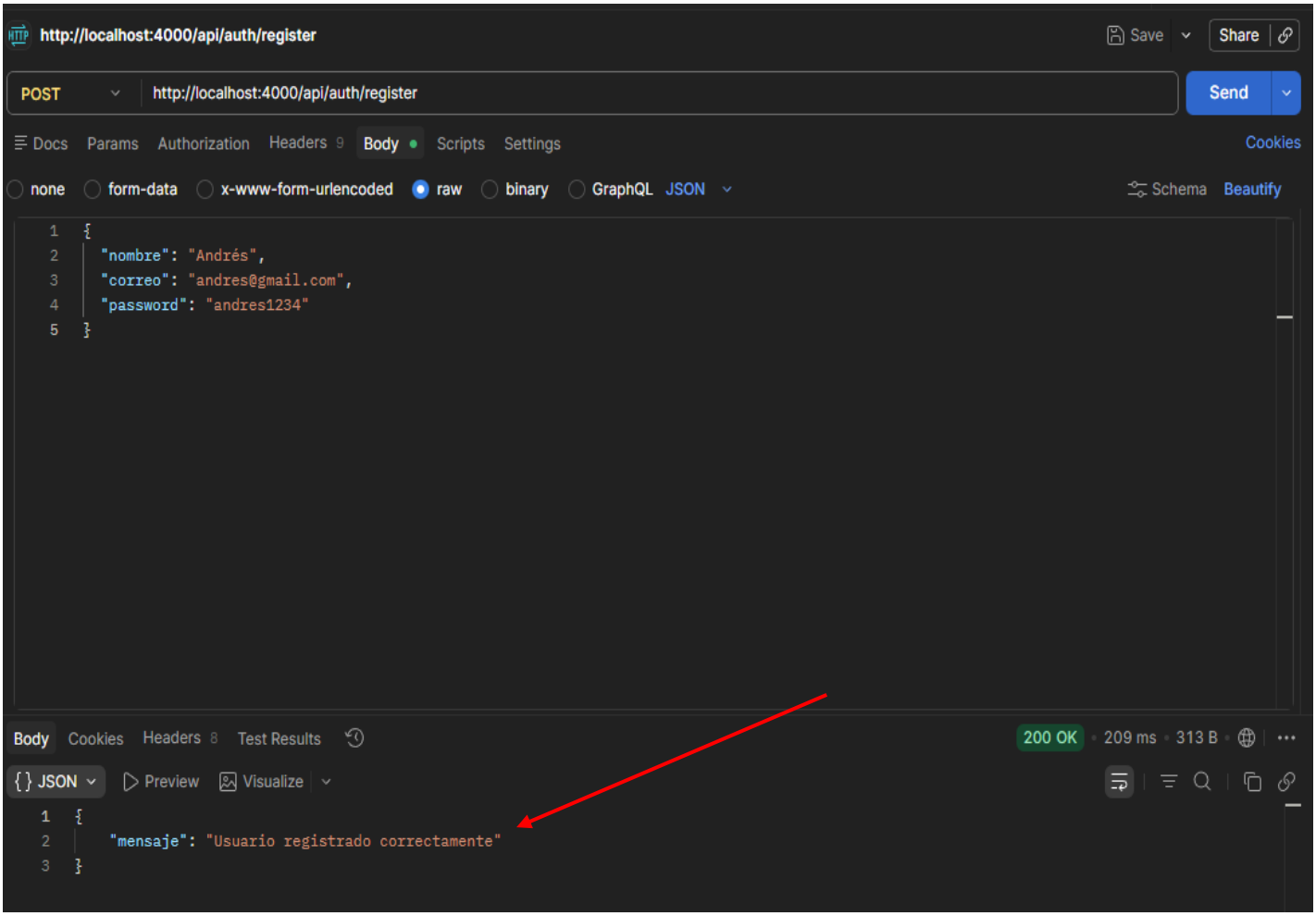
```
db.js
src > config > db.js > default
import mongoose from "mongoose";

const conectarDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log("Base de datos conectada");
  } catch (error) {
    console.error("Error al conectar la BD:", error.message);
    process.exit(1);
  }
};

export default conectarDB;
```

```
.env
.env
PORT=4000
MONGO_URI=mongodb://localhost:27017/farmacia
JWT_SECRET=secreto_super_seguro
```

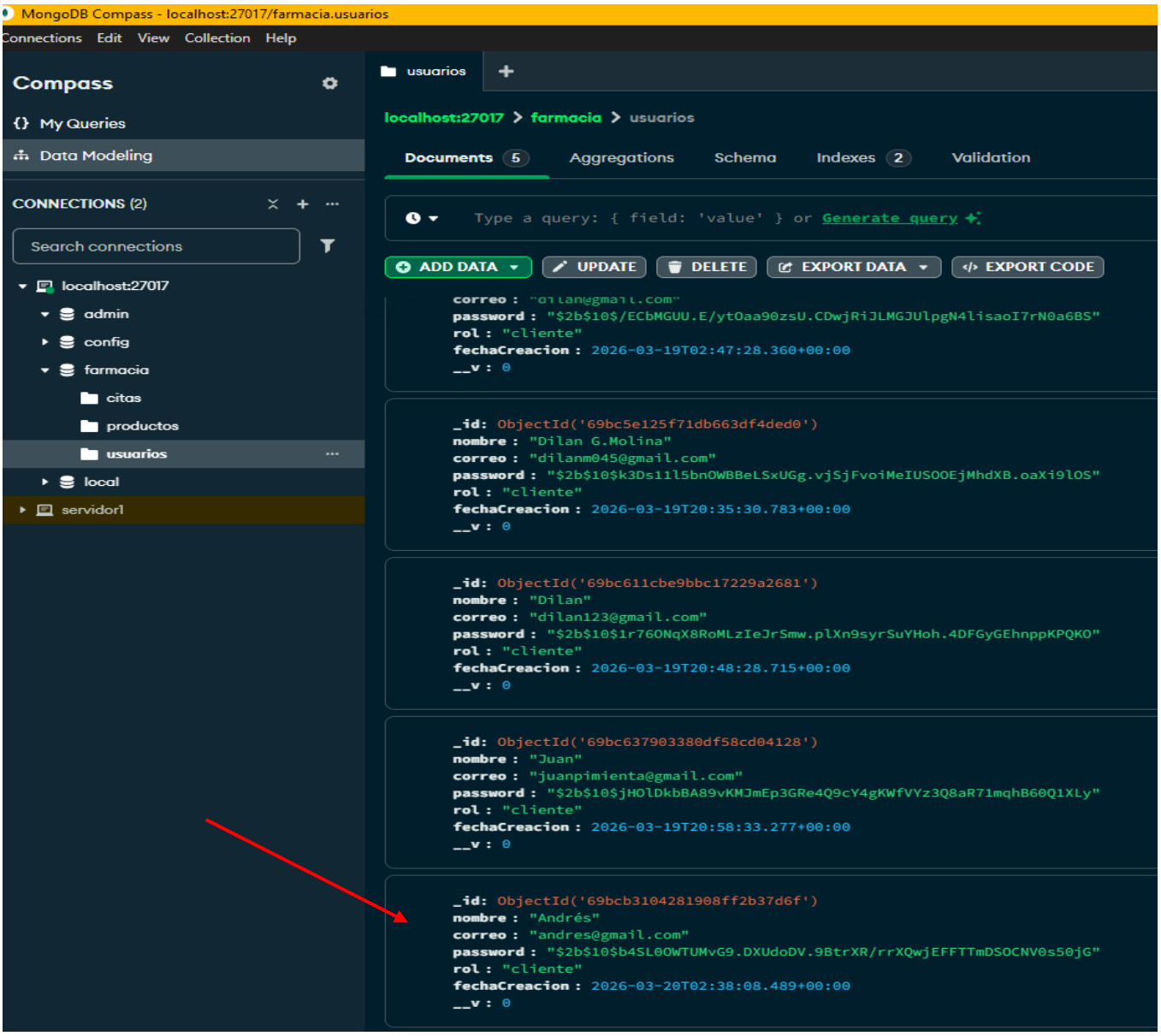
X. Pruebas realizas

10.1. Prueba de api

Prueba de registro de usuario, utilizando el método post y teniendo como req y res, un objeto json.



Prueba de que se guardo en la base de datos mongo db



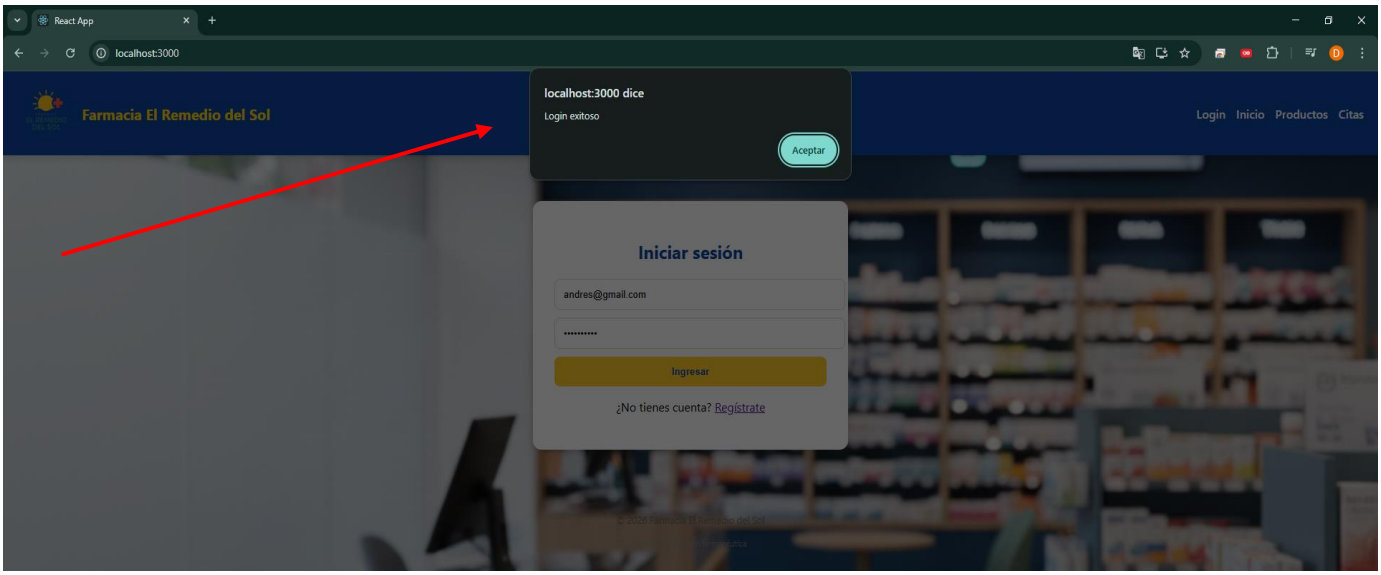
10.2. Prueba realizada en el frontend

El frontend es capaz de hacer

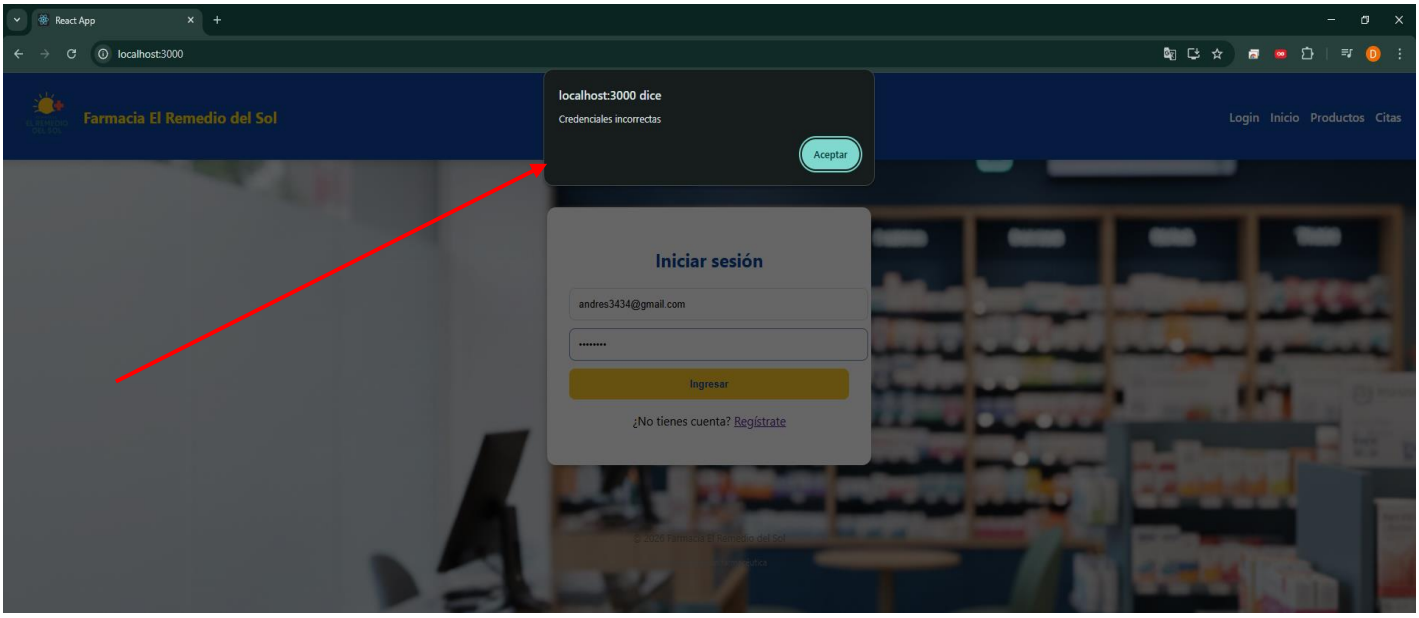
- Registro funcionando
- Login funcionando
- CRUD productos funcionando
- CRUD citas funcionando

Ejemplo

Se realiza el login con el usuario que se acaba de registrar con postman en el ejemplo anterior:



El sistema nos arroja una alerta de que el login fue exitoso, este seria un ejemplo si no realizáramos las credenciales correctas:



XI. URLs del sistema web

Api:

- /api/auth/register
- /api/auth/login
- /api/productos y /api/productos/:id
- /api/productos/:id

Frontend:

- http://localhost:3000/
- http://localhost:3000/register
- http://localhost:3000/dashboard
- http://localhost:3000/productos
- http://localhost:3000/citas

I. CONTROL DE VERSIONES

Se utilizó **Git** para el control de versiones:

Comandos utilizados en la terminal:

- git init ---- Para iniciar el repositorio
- git add . ---- Para guardar todos los cambios
- git commit -m "" --- Para guardar todos los cambios en todos los archivos conocidos en el índice
- git push origin master --- Para enviar commits (cambios) realizados localmente en la rama master

II. LIBRERÍAS Y FRAMEWORKS

Backend

- express
- mongoose
- jsonwebtoken
- cors

Frontend

- react

- react-router-dom
- axios

III. BUENAS PRÁCTICAS APLICADAS

- Separación de capas (frontend/backend)
- Uso de componentes reutilizables
- Código modular
- Nombres claros de variables
- Manejo de errores

IV. CONFIGURACIÓN DEL ENTORNO

- Node.js instalado
- MongoDB local
- Visual Studio Code
- Navegador web

V. RESULTADOS

Se logró desarrollar una aplicación funcional que permite gestionar productos y citas en una farmacia, cumpliendo con los requerimientos establecidos.

VI. CONCLUSION

El desarrollo de esta aplicación permitió aplicar conocimientos en desarrollo web full stack, integrando frontend y backend mediante API REST.

Se fortalecieron habilidades en el uso de tecnologías modernas como React, Node.js y MongoDB, así como en el manejo de control de versiones con Git.

Además, se evidenció la importancia de estructurar correctamente el código y aplicar buenas prácticas para garantizar un sistema escalable y mantenible.

Anexo link de repositorio donde anexo carpetas con todos los archivos del proyecto

https://github.com/dilanmolina4/ADSO-3118311-DILAN.git